

MEMORIA CUARTA PRACTICA: PROCESADOR DE LENGUAJE EN HASKELL

Acedo Blanco, Alvaro
Czepiel Babiarz, Daniel

26 de junio de 2026

Índice

1. Diseño del analizador léxico	2
1.1. Tokens	2
1.2. Gramática	2
1.3. Diagrama del autómata	3
1.4. Descripción de las Acciones Semánticas	3
1.5. Errores	4
2. Diseño del analizador sintáctico	5
2.1. Gramática	5
2.2. Tabla Predictiva y Demostración LL(1)	5
2.3. Gestión de Errores Sintácticos	6
3. Diseño del analizador semántico	8
3.1. Errores	10
4. Diseño de la Tabla de Símbolos	11
4.1. Organización y Ámbitos	11
4.2. Estructura de la Información	11
5. Diseño del Gestor de Errores	12
5.1. Seguimiento de Líneas	12
5.2. Estrategia de Recuperación de Errores	12
6. Anexo: Casos de prueba	13
6.1. Ficheros correctos	13
6.1.1. Fichero Correcto 1	13
6.1.2. Fichero Correcto 2	15
6.1.3. Fichero Correcto 3	18
6.1.4. Fichero Correcto 4	20
6.1.5. Fichero Correcto 5	22
6.2. Ficheros con errores	24
6.2.1. Fichero Erróneo 1	24
6.2.2. Fichero Erróneo 2	25
6.2.3. Fichero Erróneo 3	25
6.2.4. Fichero Erróneo 4	25
6.2.5. Fichero Erróneo 5	25

1. Diseño del analizador léxico

1.1. Tokens

Los tokens reconocidos por el analizador léxico, junto con sus lexemas o valores asociados, se detallan en la siguiente tabla:

Código	Lexema / Valor
entero	valor numérico
real	valor numérico
cadena	lexema
true	-
false	-
parentesisApertura	-
parentesisCierre	-
suma	-
resta	-
mayor	-
mayorIgual	-
not	-
and	-
autodecremento	-
asignacion	-
identificador	posición tabla símbolos
let	-
int	-
string	-
boolean	-
float	-
void	-
puntoComa	-
llaveApertura	-
llaveCierre	-
if	-
else	-
coma	-
function	-
read	-
return	-
write	-
eof	-

1.2. Gramática

El conjunto de terminales T de la gramática regular se define mediante la siguiente agrupación:

$$T = \begin{cases} \text{Alfanuméricos:} & \text{'a'-'z', 'A'-'Z', '0'-'9' y el carácter subrayado (_)} \\ \text{Blancos:} & \text{Espacio, Tabulador (TAB) y salto de línea (CR)} \\ \text{Operadores:} & \text{+, -, >, =, !, \&} \\ \text{Delimitadores:} & \text{(,), \{, \}, ', ', '. ', '; ', ", /} \\ \text{Especiales:} & \text{Carácter de escape (\) y fin de fichero (EOF)} \end{cases}$$

El conjunto de los no terminales es: $N = \{S, A, B, C, D, E, F, G, H, I, J, K\}$. El axioma es S y las reglas de producción P son:

$$P = \{S \rightarrow \text{EOF} \mid \text{del } S \mid dA \mid cD \mid _D \mid "E \mid (\mid) \mid + \mid -I \mid > G \mid$$

$$! \mid \&H \mid = \mid ; \mid \{ \mid \} \mid , \mid /J$$

$$\begin{aligned} A &\rightarrow dA \mid .B \mid \lambda \\ B &\rightarrow dC \\ C &\rightarrow dC \mid \lambda \\ D &\rightarrow cD \mid _D \mid dD \mid \lambda \\ E &\rightarrow c_1E \mid \backslash F \mid " \\ F &\rightarrow nE \mid tE \\ G &\rightarrow = \mid \lambda \\ H &\rightarrow \& \\ I &\rightarrow - \mid \lambda \\ J &\rightarrow /K \\ K &\rightarrow c_2K \mid CRS \mid EOF \\ \} \end{aligned}$$

Siendo: $\text{del} = \{\text{espacio, TAB, CR}\}$, $c_1 = T \setminus \{\backslash, \text{EOF, TAB, "}\}$ y $c_2 = T \setminus \{\text{EOF, TAB}\}$.

1.3. Diagrama del autómata

A partir de la gramática regular, se diseñó el siguiente Autómata Finito Determinista (AFD):

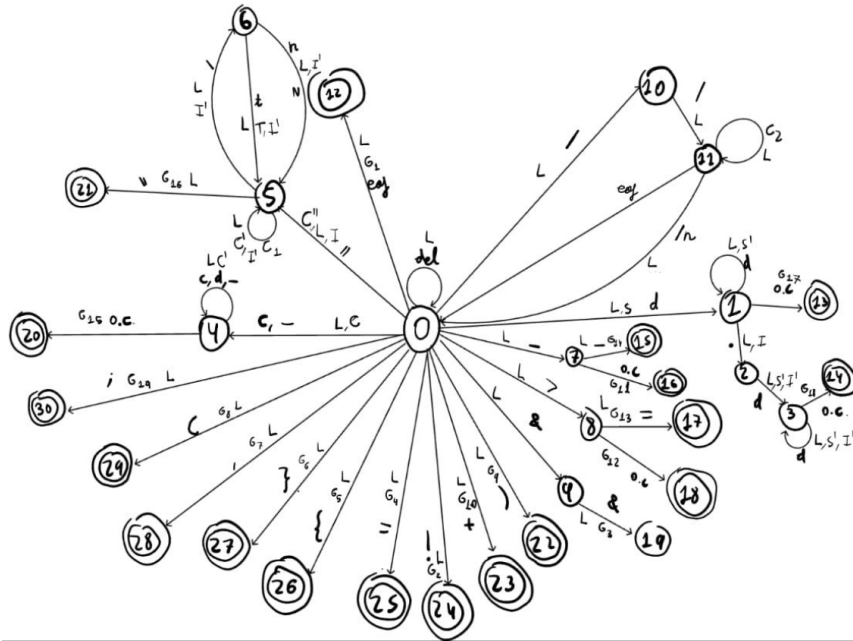


Figura 1: Autómata Finito Determinista del analizador léxico.

1.4. Descripción de las Acciones Semánticas

A continuación se detallan todas las acciones semánticas ejecutadas por el autómata durante el proceso de análisis léxico:

G1: Generar Token $\langle \text{eof}, - \rangle$

G2: Generar Token $\langle \text{not}, - \rangle$

G3: Generar Token $\langle \text{and}, - \rangle$

G4: Generar Token $\langle \text{asignacion}, - \rangle$

G5: Generar Token $\langle \text{llaveApertura}, - \rangle$

G6: Generar Token $\langle \text{llaveCierre}, - \rangle$

G7: Generar Token $\langle \text{coma}, - \rangle$

G8: Generar Token $\langle \text{parentesisApertura}, - \rangle$

G9: Generar Token $\langle \text{parentesisCierre}, - \rangle$

G10: Generar Token $\langle \text{suma}, - \rangle$

G11: Generar Token $\langle \text{resta}, - \rangle$

G12: Generar Token $\langle \text{mayor}, - \rangle$

G13: Generar Token $\langle \text{mayorIgual}, - \rangle$

G14: Generar Token $\langle \text{autodecremento}, - \rangle$

G15: **if** $p \neq \text{null}$ **then**
 Generar Token $\langle \text{PR_p}, - \rangle$
 else
 $p = \text{BuscarTS}(\text{lexema})$
 if $p == \text{null}$ **then**
 $p = \text{InsertarTS}(\text{lexema})$
 Generar Token $\langle \text{identificador}, p \rangle$

G16: **if** $\text{cont} < 65$ **then**
 Generar Token $\langle \text{cadena}, \text{lexema} \rangle$
 else
 Error(28)

G17: **if** $2^{15} > \text{num}$ **then**
 Generar Token $\langle \text{entero}, \text{num} \rangle$
 else
 Error(21)

G18: $\text{num} = \text{num} \times 10^{-\text{cont}}$
 if $\text{num} < \text{NUM_MAX}$ **then**
 Generar Token $\langle \text{real}, \text{num} \rangle$
 else
 Error(24)

G19: Generar Token $\langle \text{puntoComa}, - \rangle$

L: Leer el siguiente caracter

C: $\text{lexema} = \text{caracter}$ (Inicializa el buffer del lexema con el carácter actual)

C': $\text{lexema} = \text{lexema} + \text{caracter}$ (Concatena el carácter actual al final del lexema)

C'': $\text{lexema} = ''$; $\text{cont} = 0$ (Limpia el lexema acumulado y reinicia el contador de longitud)

N: $\text{lexema} = \text{lexema} + '\backslash n'$

1.5. Errores

El analizador léxico detecta y gestiona las siguientes anomalías:

- Aparición de un carácter desconocido que no pertenece al alfabeto del lenguaje.
- Uso incorrecto o inacabado de comentarios (del tipo `//`).
- Uso incompleto del operador lógico **and** (un solo **&**).
- Falta de terminación lógica de las cadenas de texto antes de un salto de línea.
- Uso de secuencias de escape no válidas en literales de cadena.
- Números reales mal formados (ej. falta de parte decimal tras el punto).
- Exceso de tamaño para los literales numéricos o exceso de longitud en cadenas.

2. Diseño del analizador sintáctico

2.1. Gramática

Para el desarrollo del analizador sintáctico se ha diseñado una gramática en contexto formalizada para un análisis descendente tabular ($LL(1)$), definida mediante la siguiente cuádrupla:

El conjunto de símbolos no terminales (V_N) es:

$$V_N = \{A, B, C, D, E, E', F, G, H, I, K, L, O, P, Q, R, R', S, T, U, U', V, W, X, Z\}$$

El conjunto de símbolos terminales (V_T) corresponde a los tokens identificados por el analizador léxico:

$V_T = \{\text{id, entero, real, cadena, true, false, let, int, float, boolean, string, void, if, else, function, read, write, return, \&\&, >, >=, +, -, !}\}$

El axioma inicial es:

$$\text{Axioma} = P$$

El conjunto completo de producciones (P) se numera secuencialmente a continuación:

- | | |
|--|--|
| (1) $E \rightarrow RE'$ | (32) $L \rightarrow EQ$ |
| (2) $E' \rightarrow \&\&RE'$ | (33) $L \rightarrow \lambda$ |
| (3) $E' \rightarrow \lambda$ | (34) $Q \rightarrow, EQ$ |
| (4) $R \rightarrow UR'$ | (35) $Q \rightarrow \lambda$ |
| (5) $R' \rightarrow > UR'$ | (36) $X \rightarrow E$ |
| (6) $R' \rightarrow >= UR'$ | (37) $X \rightarrow \lambda$ |
| (7) $R' \rightarrow \lambda$ | (38) $B \rightarrow \text{if } (E)Z$ |
| (8) $U \rightarrow VU'$ | (39) $Z \rightarrow S$ |
| (9) $U' \rightarrow +VU'$ | (40) $Z \rightarrow \{C\}I$ |
| (10) $U' \rightarrow -VU'$ | (41) $I \rightarrow \text{else } D$ |
| (11) $U' \rightarrow \lambda$ | (42) $I \rightarrow \lambda$ |
| (12) $V \rightarrow +W$ | (43) $D \rightarrow \text{if } (E)\{C\}I$ |
| (13) $V \rightarrow -W$ | (44) $D \rightarrow \{C\}$ |
| (14) $V \rightarrow !W$ | (45) $B \rightarrow \text{let } T \text{ id;}$ |
| (15) $V \rightarrow - - W$ | (46) $B \rightarrow S$ |
| (16) $V \rightarrow W$ | (47) $T \rightarrow \text{int}$ |
| (17) $W \rightarrow \text{id } O$ | (48) $T \rightarrow \text{float}$ |
| (18) $W \rightarrow (E)$ | (49) $T \rightarrow \text{boolean}$ |
| (19) $W \rightarrow \text{entero}$ | (50) $T \rightarrow \text{string}$ |
| (20) $W \rightarrow \text{real}$ | (51) $F \rightarrow \text{function } H \text{ id } (A)\{C\}$ |
| (21) $W \rightarrow \text{cadena}$ | (52) $H \rightarrow T$ |
| (22) $W \rightarrow \text{true}$ | (53) $H \rightarrow \text{void}$ |
| (23) $W \rightarrow \text{false}$ | (54) $A \rightarrow T \text{ id } K$ |
| (24) $O \rightarrow (L)$ | (55) $A \rightarrow \text{void}$ |
| (25) $O \rightarrow \lambda$ | (56) $K \rightarrow, T \text{ id } K$ |
| (26) $S \rightarrow \text{id } G;$ | (57) $K \rightarrow \lambda$ |
| (27) $S \rightarrow \text{write } E;$ | (58) $C \rightarrow BC$ |
| (28) $S \rightarrow \text{read id;}$ | (59) $C \rightarrow \lambda$ |
| (29) $S \rightarrow \text{return } X;$ | (60) $P \rightarrow BP$ |
| (30) $G \rightarrow = E$ | (61) $P \rightarrow FP$ |
| (31) $G \rightarrow (L)$ | (62) $P \rightarrow \lambda$ |

2.2. Tabla Predictiva y Demostración $LL(1)$

Para la validación formal del analizador sintáctico, se calcularon los conjuntos de *First* (Primero) y *Follow* (Siguiendo) de cada regla de producción. La tabla resultante demuestra empíricamente que la gramática planteada es estrictamente $LL(1)$, debido a la total ausencia de conflictos de tipo desplazamiento-reducción o reducción-reducción; es decir, no coincide más de una regla sintáctica en ninguna de las casillas predictivas.

Se adjunta la tabla al final de esta sección.

2.3. Gestión de Errores Sintácticos

El analizador sintáctico realiza un control de flujo estructurado para interceptar y registrar anomalías gramaticales sin abortar el proceso global. Conceptualmente, las inconsistencias se clasifican en dos naturalezas:

1. **Discordancia de Terminales:** Se produce cuando el símbolo en la cima de la pila de derivación es un terminal concreto, pero no se corresponde lógicamente con el token proporcionado en ese instante por el analizador léxico.
2. **Celdas Vacías en la Tabla Predictiva:** Ocurre cuando la cima de la pila contiene un símbolo no terminal, pero al cruzarlo con el token actual del flujo de entrada, la matriz predictiva devuelve una transición indeterminada o nula (**Nothing**), denotando una estructura de sentencia inválida.

Cuadro 1: Tabla de Análisis Sintáctico

[illegible]

3. Diseño del analizador semántico

Para el diseño del analizador semántico se ha definido el siguiente Esquema de Traducción (EdT), detallando las acciones semánticas asociadas a cada regla de producción para la comprobación de tipos y generación de la tabla de símbolos.

#	Producción	Acciones Semánticas
1	$E \rightarrow R E_1$	$E.tipo = \text{if}(E_1.tipo == \text{void} \vee R.tipo == E_1.tipo)$ then $R.tipo$ else tipo_error {AUX -= 2}
2	$E_1 \rightarrow \&\& R E_{1_1}$	$E_1.tipo = \text{if}(R.tipo == \text{logico} \wedge E_{1_1}.tipo \in \{\text{logico}, \text{void}\})$ then $R.tipo$ else tipo_error {AUX -= 3}
3	$E_1 \rightarrow \lambda$	$E_1.tipo = \text{void}$ {AUX -= 0}
4	$R \rightarrow U R_1$	$R.tipo = \text{if}(R_1.tipo == \text{void})$ then $U.tipo$ else if($U.tipo == R_1.tipo$) then logico else tipo_error {AUX -= 2}
5	$R_{1_1} \rightarrow > U R_{1_2}$	$R_{1_1}.tipo = \text{if}(U.tipo \in \{\text{real}, \text{entero}\} \wedge R_{1_2} == \text{void})$ then $U.tipo$ else tipo_error {AUX -= 3}
6	$R_{1_1} \rightarrow \geq U R_{1_2}$	$R_{1_1}.tipo = \text{if}(U.tipo \in \{\text{real}, \text{entero}\} \wedge R_{1_2} == \text{void})$ then $U.tipo$ else tipo_error {AUX -= 3}
7	$R_1 \rightarrow \lambda$	$R_1.tipo = \text{void}$ {AUX -= 0}
8	$U \rightarrow V U_1$	$U.tipo = \text{if}(U_1.tipo == \text{void} \vee V.tipo == U_1.tipo)$ then $V.tipo$ else tipo_error {AUX -= 2}
9	$U_1 \rightarrow + V U_{1_1}$	$U_1.tipo = \text{if}((V.tipo \in \{\text{entero}, \text{real}\} \wedge U_{1_1}.tipo == \text{void}) \vee V.tipo == U_{1_1}.tipo)$ then $V.tipo$ else tipo_error {AUX -= 3}
10	$U_1 \rightarrow - V U_{1_1}$	$U_1.tipo = \text{if}((V.tipo \in \{\text{entero}, \text{real}\} \wedge U_{1_1}.tipo == \text{void}) \vee V.tipo == U_{1_1}.tipo)$ then $V.tipo$ else tipo_error {AUX -= 3}
11	$U_1 \rightarrow \lambda$	$U_1.tipo = \text{void}$ {AUX -= 0}
12	$V \rightarrow + W$	$V.tipo = \text{if}(W.tipo \in \{\text{entero}, \text{real}\})$ then $W.tipo$ else tipo_error {AUX -= 2}
13	$V \rightarrow - W$	$V.tipo = \text{if}(W.tipo \in \{\text{entero}, \text{real}\})$ then $W.tipo$ else tipo_error {AUX -= 2}

Continúa en la siguiente página...

#	Producción	Acciones Semánticas
14	$V \rightarrow ! W$	$V.tipo = \text{if}(W.tipo == \text{lógico}) \text{ then } tipo_logico$ $\text{else } tipo_error$ $\{AUX \ -= \ 2\}$
15	$V \rightarrow - - id$	$V.tipo = \text{if}(\text{buscarTipo}(id.pos) \in \{\text{real}, \text{entero}\})$ $\text{then } buscarTipo(id.pos)$ $\text{else } tipo_error$ $\{AUX \ -= \ 2\}$
16	$V \rightarrow W$	$V.tipo = W.tipo$ $\{AUX \ -= \ 1\}$
17	$W \rightarrow id O$	$W.tipo = \text{if}(O.tipo == \text{void}) \text{ then } buscarTipo(id.pos)$ $\text{else if}(\text{buscarParam}(id.pos) == O.parametros)$ $\text{then } buscarTipoRet(id.pos)$ $\text{else } tipo_error$ $\{AUX \ -= \ 2\}$
18	$W \rightarrow (E)$	$W.tipo = E.tipo$ $\{AUX \ -= \ 3\}$
19	$W \rightarrow \text{entero}$	$W.tipo = \text{entero}$ $\{AUX \ -= \ 1\}$
20	$W \rightarrow \text{real}$	$W.tipo = \text{real}$ $\{AUX \ -= \ 1\}$
21	$W \rightarrow \text{cadena}$	$W.tipo = \text{cadena}$ $\{AUX \ -= \ 1\}$
22	$W \rightarrow \text{true}$	$W.tipo = \text{lógico}$ $\{AUX \ -= \ 1\}$
23	$W \rightarrow \text{false}$	$W.tipo = \text{lógico}$ $\{AUX \ -= \ 1\}$
24	$O \rightarrow (L)$	$O.tipo = tipo_funcion$ $O.parametros = L.parametros$ $O.numParam = L.numeroParametros$ $\{AUX \ -= \ 3\}$
25	$O \rightarrow \lambda$	$O.tipo = \text{void}$ $\{AUX \ -= \ 0\}$
26	$S \rightarrow id G ;$	$S.tipo = \text{if}(\text{buscarTipo}(id.pos) == G.tipo == tipo_funcion)$ $\text{then if}(\text{buscarParam}(id.pos) \neq G.parametros) \text{ tipo_error}$ $\text{else if}(\text{buscarTipo}(id.pos) \neq G.tipo) \text{ then } tipo_error$ $\{AUX \ -= \ 3\}$
27	$S \rightarrow \text{write } E ;$	$S.tipo = \text{if}(E.tipo \notin \{\text{entero}, \text{real}, \text{cadena}\}) \text{ then } tipo_error$ $S.tipoRet = \text{void}$ $\{AUX \ -= \ 3\}$
28	$S \rightarrow \text{read } id ;$	$S.tipo = \text{if}(\text{buscarTipoTS}(id.pos) \notin \{\text{entero}, \text{real}, \text{cadena}\})$ $\text{then } tipo_error$ $S.tipoRet = \text{void}$ $\{AUX \ -= \ 3\}$
29	$S \rightarrow \text{return } X ;$	$S.tipo = \text{if}(X.tipo \neq S.tipoRet) \text{ then } tipo_error$ $\{AUX \ -= \ 3\}$
30	$G \rightarrow = E$	$G.tipo = E.tipo$ $\{AUX \ -= \ 2\}$
31	$G \rightarrow (L)$	$G.tipo = tipo_funcion$ $G.parametros = L.parametros, G.numParam =$ $L.numParametros$ $\{AUX \ -= \ 3\}$
Continúa en la siguiente página...		

#	Producción	Acciones Semánticas
32	$L \rightarrow E Q$	$L.param = E.tipo \times Q.param, L.numP = 1 + Q.numP$ {AUX -= 2}
33	$L \rightarrow \lambda$	$L.param = \text{void}, L.numP = 0$ {AUX -= 0}
34	$Q_1 \rightarrow, E Q_2$	$Q_1.param = E.tipo \times Q_2.param, Q_1.numP = 1 + Q_2.numP$ {AUX -= 3}
35	$Q \rightarrow \lambda$	$Q.param = \text{void}, Q.numP = 0$ {AUX -= 0}
38, 39	$B \rightarrow \text{if} (E) Z$	$B.tipo = \text{if}(E.tipo == \text{logico}) \text{ then } tipo_error$ $B.tipoRet = Z.tipoRet$ {AUX -= 5}
51, 52	$B \rightarrow \text{let } T \text{ id};$	$zonaDecl = \text{true}$ $\text{añadirTipo}(\text{id.pos}, T.tipo)$ $\text{añadirDespl}(\text{id.pos}, \text{despl})$ $\text{desplazamiento} += T.tamaño$ $zonaDecl = \text{false}$ {AUX -= 4}
58-60	$F \rightarrow \text{function } H \text{ id} (A) \{ C \}$	$zonaDecl = \text{true}, \text{añadeTipo}(\text{id.pos}, \text{tipo_func})$ $\text{añadeTipoRet}(\text{id.pos}, H.tipo), C.tipoRet = H.tipo$ $\text{añadeEtiq}(\text{id.pos}, \text{nuevaEtiq}())$ $TSF = \text{crearTabla}(), \text{despl} = 0, zonaDecl = \text{false}$ $\text{añadeParam}(\text{id.pos}, A.param), \text{liberarTabla}(TSF)$ {AUX -= 9}
68, 69	$PP \rightarrow P$	$TS = \text{crearTabla}(), \text{desplazamiento} = 0$ $\text{liberarTabla}(TS)$ {AUX -= 1}

3.1. Errores

El analizador semántico detecta y gestiona los siguientes errores de tipado e incoherencias de contexto:

- Uso de tipos incompatibles en expresiones lógicas y relacionales (ej. operadores `and`, `>`, `>=`).
- Operaciones aritméticas sin conversión implícita de tipos (mezcla de enteros y reales sin coerción explícita).
- Invocación de funciones con número o tipo incorrecto de parámetros en comparación con su declaración.
- Asignación de valores a variables de distinto tipo.
- Utilización de las sentencias `write` y `read` con variables de tipos no permitidos (solo se admite entero, real o cadena).
- Discordancia entre el tipo devuelto por la sentencia `return` y el tipo de retorno declarado en la firma de la función.
- Evaluación de sentencias de control (como el condicional `if`) donde la expresión interna no se resuelve a un tipo lógico.

4. Diseño de la Tabla de Símbolos

Para la gestión de los identificadores del programa se ha diseñado una Tabla de Símbolos estructurada en dos niveles de ámbito: global y local. Esta decisión permite dar soporte a la declaración de variables dentro y fuera de las funciones, así como al sombreado de variables.

4.1. Organización y Ámbitos

- **Tabla Global:** Almacena las variables declaradas en el flujo principal del programa, así como las firmas de las funciones. Sus elementos se indexan lógicamente con valores enteros positivos (0, 1, 2, ...).
- **Tabla Local:** Se crea de forma dinámica al entrar en el contexto de una función (tras leer la palabra reservada `function`) y se destruye al salir de la misma. Para diferenciar sus entradas de las globales de forma eficiente, los identificadores locales se indexan mediante valores enteros negativos (-1, -2, -3, ...).

4.2. Estructura de la Información

La política de acceso y resolución de nombres se basa en una búsqueda lineal, consultando siempre en primer lugar el ámbito local (si existe) y, de no encontrar coincidencia, procediendo al ámbito global.

Cada registro en la tabla almacena los siguientes atributos actualizados por el analizador semántico:

- **Lexema:** La cadena de texto que da nombre al identificador.
- **Tipo de dato:** Entero, real, lógico, cadena, void o función.
- **Desplazamiento (*Offset*):** Dirección relativa de memoria calculada a partir del tamaño en bytes del tipo de dato asociado (ej. entero = 2 bytes, real = 4 bytes, cadena = 64 bytes).
- **Atributos de función:** Si el identificador corresponde a una función, se almacena adicionalmente su tipo de retorno, el número total de parámetros esperados y una lista ordenada con los tipos exactos de dichos parámetros.

5. Diseño del Gestor de Errores

El gestor de errores es un módulo transversal invocado por los analizadores léxico, sintáctico y semántico ante cualquier anomalía en el código fuente.

5.1. Seguimiento de Líneas

Para proporcionar diagnósticos precisos que permitan al programador localizar los fallos, el procesador mantiene un contador de línea actual. Este contador es incrementado exclusivamente por el analizador léxico cada vez que detecta el delimitador de salto de línea. Cuando el gestor registra un error, este se asocia de manera automática a la línea en curso.

5.2. Estrategia de Recuperación de Errores

Se ha tomado la decisión de diseño de implementar un mecanismo de recuperación de errores en lugar de abortar la compilación o detener el programa ante el primer fallo detectado. El objetivo es analizar la totalidad del fichero fuente y emitir un informe completo en una sola pasada.

El funcionamiento lógico se estructura de la siguiente manera:

- **Detección y Registro:** Cuando un analizador detecta una estructura inválida (ej. un carácter extraño, un token no esperado o un tipo semántico incompatible), se genera una alerta con su código de error, descripción y línea. Esta alerta se concatena en el registro interno del gestor.
- **Sincronización:** En lugar de colapsar, el sistema descarta la cadena conflictiva o reinicia las estructuras de análisis temporales (como la pila sintáctica o el lexema acumulado) hasta encontrar el siguiente punto de sincronización lógico y seguro (por ejemplo, un delimitador de final de sentencia ; o el cierre de un bloque }). Esto permite retomar la evaluación del resto del código.
- **Inconvenientes (Errores en Cascada):** Como contrapartida a la recuperación sistemática, esta técnica genera en ocasiones errores derivados o falsos positivos. Por ejemplo, si una cadena de texto no se cierra correctamente con comillas dobles, el analizador léxico absorberá el resto de caracteres de su misma línea, lo que provocará casi con seguridad que el analizador sintáctico emita errores por falta de terminadores lógicos (como el punto y coma) que quedaron engullidos. Esta es una decisión consciente y un estándar adoptado por los compiladores modernos, asumiendo que informar de múltiples fallos útiles compensa la aparición ocasional de falsos positivos en cascada.

6. Anexo: Casos de prueba

6.1. Ficheros correctos

6.1.1. Fichero Correcto 1

```
1 let int contador;  
2 let float precio;  
3 let boolean activo;  
4 contador = 100;  
5 precio = 9.99;  
6 activo = true;  
7 function int duplicar(int n) {  
8     return n + n;  
9 }  
10 write duplicar(contador);  
11 if (activo) {  
12     write precio;  
13 }
```

fichero1.txt

Listado de Tokens:

```
1 <let,>  
2 <int,>  
3 <identificador,0>  
4 <puntoComa,>  
5 <let,>  
6 <float,>  
7 <identificador,1>  
8 <puntoComa,>  
9 <let,>  
10 <boolean,>  
11 <identificador,2>  
12 <puntoComa,>  
13 <identificador,0>  
14 <asignacion,>  
15 <entero,100>  
16 <puntoComa,>  
17 <identificador,1>  
18 <asignacion,>  
19 <real,9.99>  
20 <puntoComa,>  
21 <identificador,2>  
22 <asignacion,>  
23 <true,>  
24 <puntoComa,>  
25 <function,>  
26 <int,>  
27 <identificador,3>  
28 <parentesisApertura,>  
29 <int,>  
30 <identificador,-1>  
31 <parentesisCierre,>  
32 <llaveApertura,>  
33 <return,>  
34 <identificador,-1>  
35 <suma,>  
36 <identificador,-1>  
37 <puntoComa,>  
38 <llaveCierre,>  
39 <write,>  
40 <identificador,3>  
41 <parentesisApertura,>  
42 <identificador,0>  
43 <parentesisCierre,>  
44 <puntoComa,>  
45 <if,>  
46 <parentesisApertura,>
```

```

47 <identificador,2>
48 <parentesisCierre,>
49 <llaveApertura,>
50 <write,>
51 <identificador,1>
52 <puntoComa,>
53 <llaveCierre,>
54 <eof,>

```

Tabla de Símbolos:

```

1  CONTENIDO DE LA TABLA # 1 :
2  * LEXEMA : 'contador'
3    +Tipo: entero
4    +Desplazamiento: 0
5  * LEXEMA : 'precio'
6    +Tipo: real
7    +Desplazamiento: 2
8  * LEXEMA : 'activo'
9    +Tipo: lógico
10   +Desplazamiento: 6
11 * LEXEMA : 'duplicar'
12   +Tipo: función
13   +Desplazamiento: 0
14   +TipoRet: entero
15   +NumParametros: 1
16   +Parametros: entero
17
18 CONTENIDO DE LA TABLA # 2 :
19 * LEXEMA : 'n'
20   +Tipo: entero
21   +Desplazamiento: 0

```

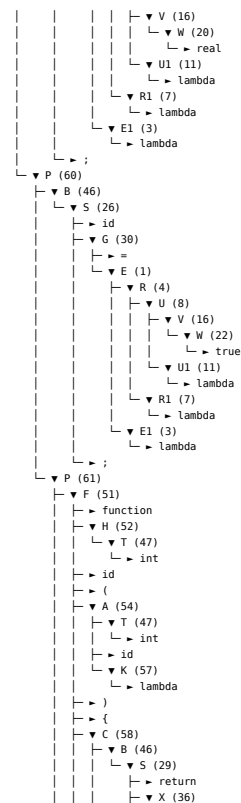
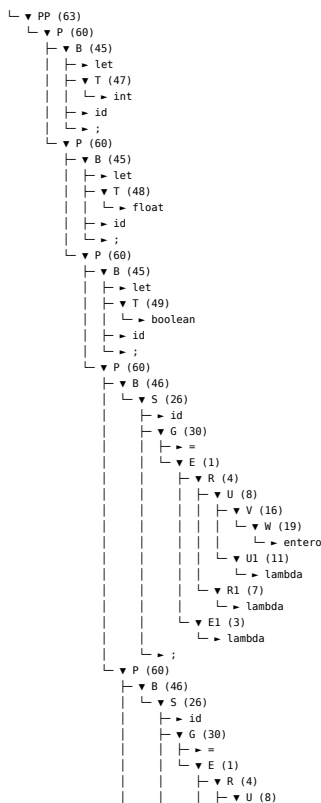
Secuencia de Parse:

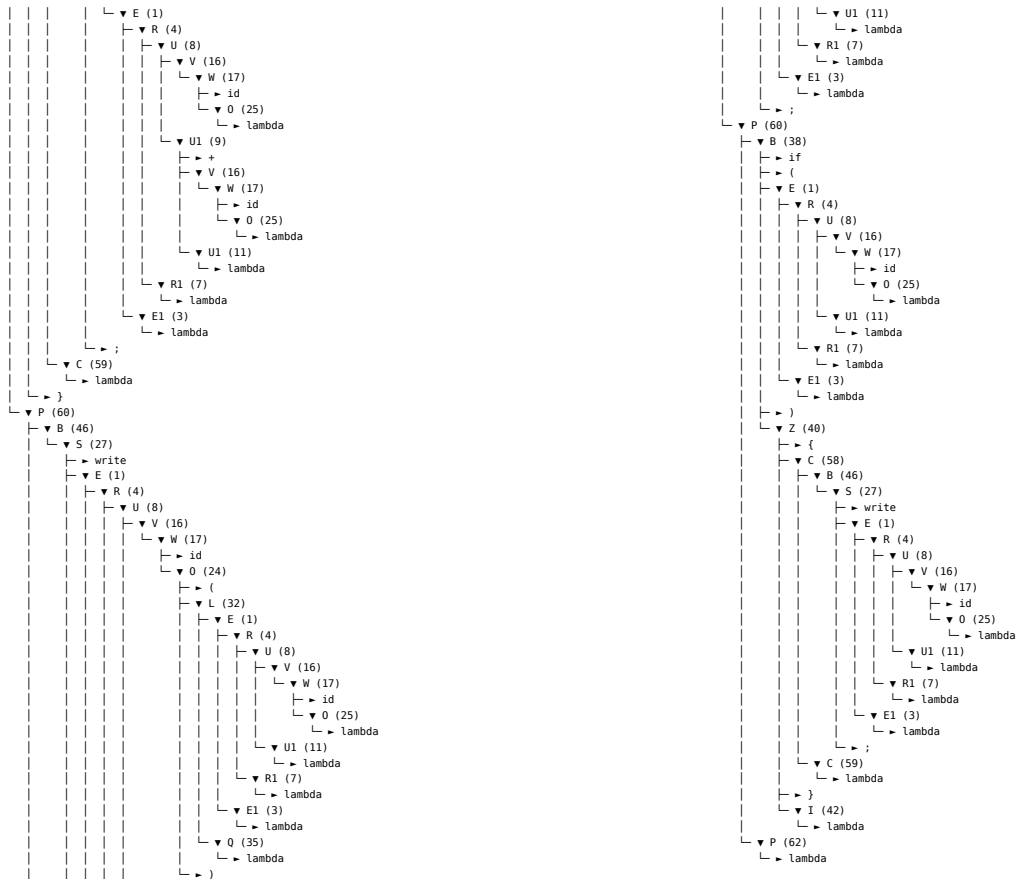
```

1  Descendente
2  63 60 45 47 60 45 48 60 45 49 60 46 26 30 1 4 8 16 19 11 7 3 60 46 26 30 1 4 8 16 20 11 7 3 60 46 26 30 1 4 8 16 22 11 7
   3 61 51 52 47 54 47 57 58 46 29 36 1 4 8 16 17 25 9 16 17 25 11 7 3 59 60 46 27 1 4 8 16 17 24 32 1 4 8 16 17 25
   11 7 3 35 11 7 3 60 38 1 4 8 16 17 25 11 7 3 40 58 46 27 1 4 8 16 17 25 11 7 3 59 42 62

```

Árbol de Análisis Sintáctico:





6.1.2. Fichero Correcto 2

```

1 let int resultado;
2 function int maximo(int a, int b) {
3     if (a > b)
4         return a;
5     return b;
6 }
7 function int sumarMaximo(int x, int y, int z) {
8     return maximo(x, y) + z;
9 }
10 resultado = sumarMaximo(3, 7, 10);
11 write resultado;

```

fichero2.txt

Listado de Tokens:

```

1 <let,>
2 <int,>
3 <identificador,0>
4 <puntoComa,>
5 <function,>
6 <int,>
7 <identificador,1>
8 <parentesisApertura,>
9 <int,>
10 <identificador,-1>
11 <coma,>
12 <int,>
13 <identificador,-2>
14 <parentesisCierre,>
15 <llaveApertura,>
16 <if,>
17 <parentesisApertura,>
18 <identificador,-1>
19 <mayor,>
20 <identificador,-2>

```

```

21 <parentesisCierre,>
22 <return,>
23 <identificador,-1>
24 <puntoComa,>
25 <return,>
26 <identificador,-2>
27 <puntoComa,>
28 <llaveCierre,>
29 <function,>
30 <int,>
31 <identificador,2>
32 <parentesisApertura,>
33 <int,>
34 <identificador,-1>
35 <coma,>
36 <int,>
37 <identificador,-2>
38 <coma,>
39 <int,>
40 <identificador,-3>
41 <parentesisCierre,>
42 <llaveApertura,>
43 <return,>
44 <identificador,1>
45 <parentesisApertura,>
46 <identificador,-1>
47 <coma,>
48 <identificador,-2>
49 <parentesisCierre,>
50 <suma,>
51 <identificador,-3>
52 <puntoComa,>
53 <llaveCierre,>
54 <identificador,0>
55 <asignacion,>
56 <identificador,2>
57 <parentesisApertura,>
58 <entero,3>
59 <coma,>
60 <entero,7>
61 <coma,>
62 <entero,10>
63 <parentesisCierre,>
64 <puntoComa,>
65 <write,>
66 <identificador,0>
67 <puntoComa,>
68 <eof,>

```

Tabla de Símbolos:

```

1 CONTENIDO DE LA TABLA # 1 :
2 * LEXEMA : 'resultado'
3   +Tipo: entero
4   +Desplazamiento: 0
5 * LEXEMA : 'maximo'
6   +Tipo: función
7   +Desplazamiento: 0
8   +TipoRet: entero
9   +NumParametros: 2
10  +Parametros: entero, entero
11 * LEXEMA : 'sumarMaximo'
12   +Tipo: función
13   +Desplazamiento: 0
14   +TipoRet: entero
15   +NumParametros: 3
16   +Parametros: entero, entero, entero
17
18 CONTENIDO DE LA TABLA # 2 :

```



```

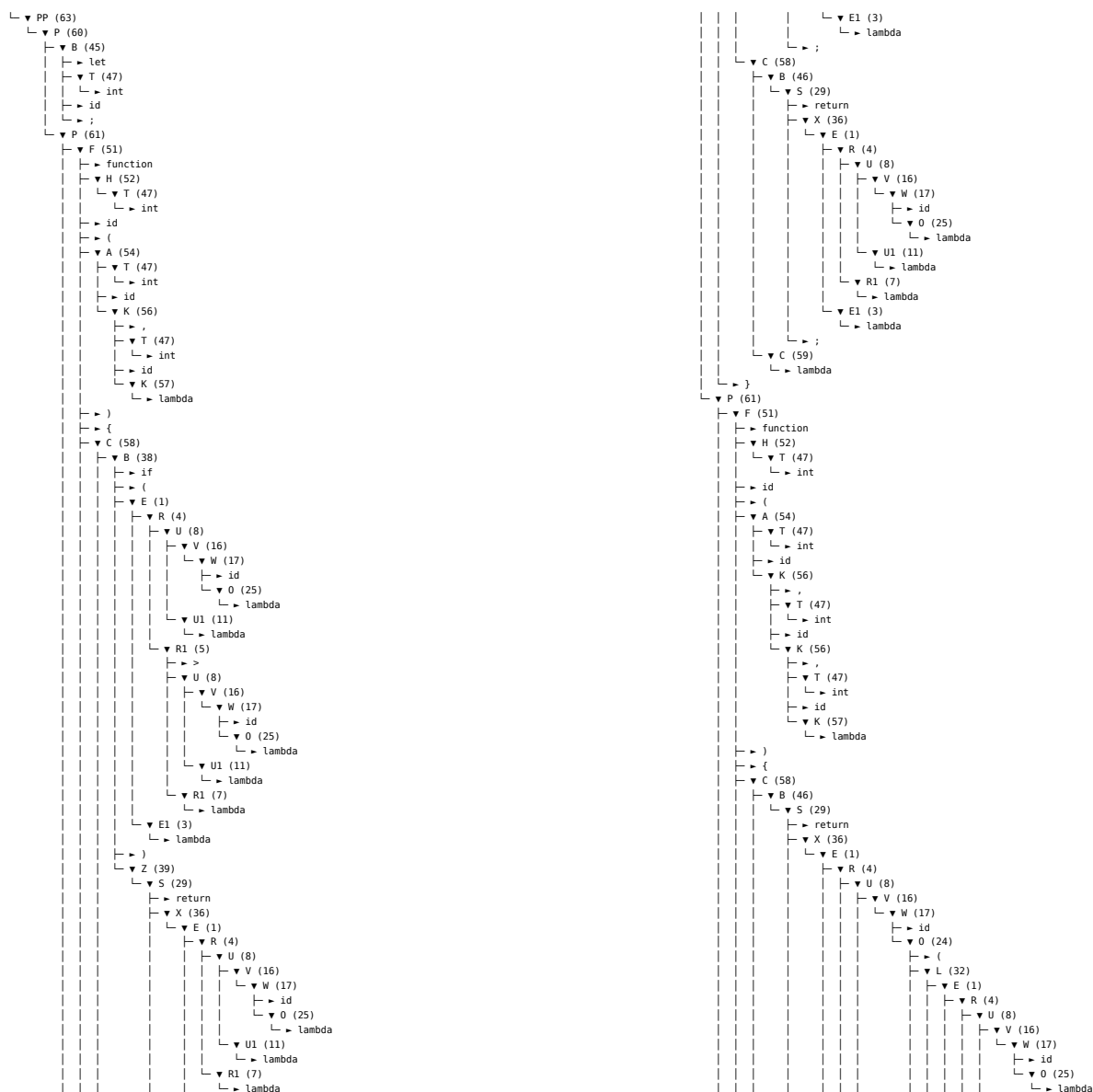
19 * LEXEMA : 'a'
20     +Tipo: entero
21     +Desplazamiento: 0
22 * LEXEMA : 'b'
23     +Tipo: entero
24     +Desplazamiento: 2
25
26 CONTENIDO DE LA TABLA # 3 :
27 * LEXEMA : 'x'
28     +Tipo: entero
29     +Desplazamiento: 0
30 * LEXEMA : 'y'
31     +Tipo: entero
32     +Desplazamiento: 2
33 * LEXEMA : 'z'
34     +Tipo: entero
35     +Desplazamiento: 4

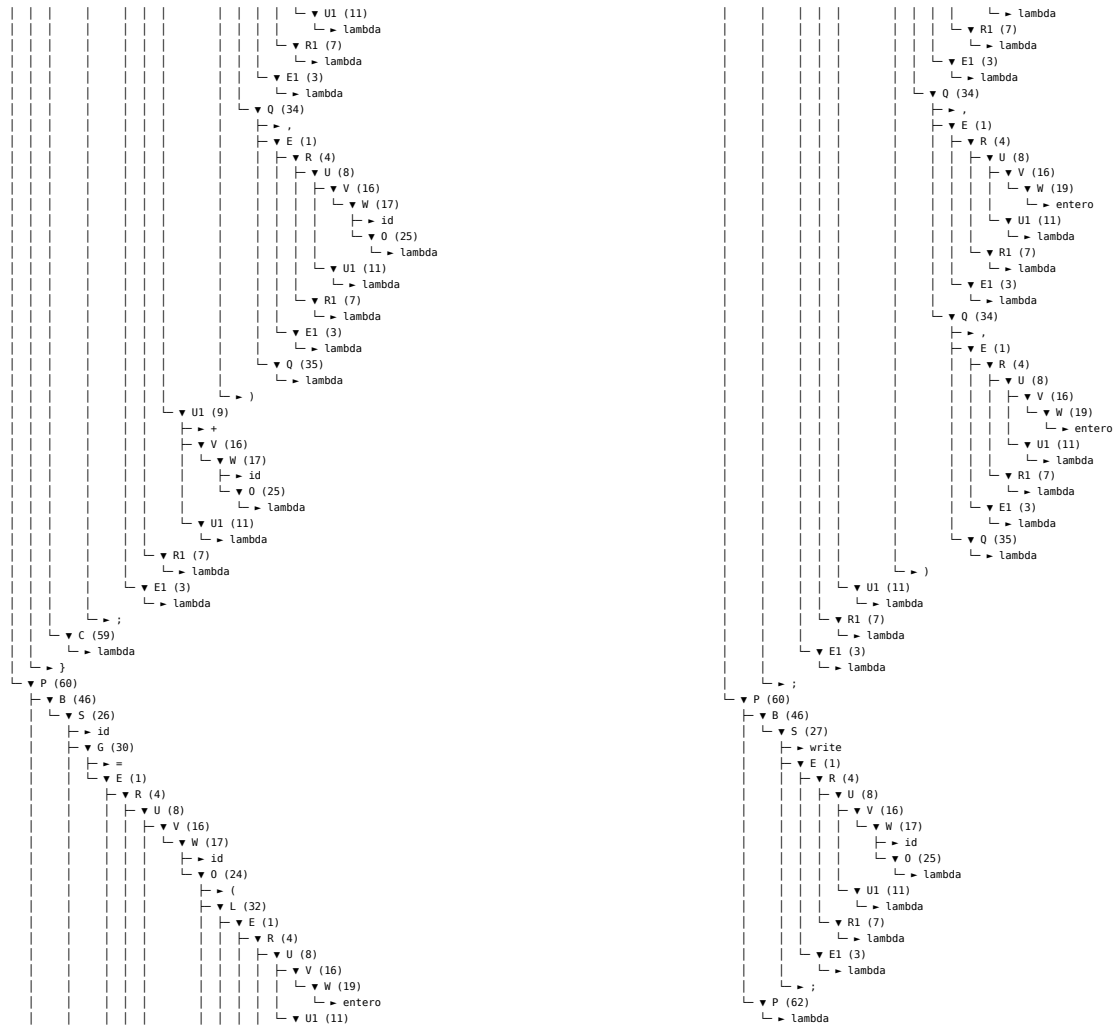
```

Secuencia de Parse:

1	Descendente
2	63 60 45 47 61 51 52 47 54 47 56 47 57 58 38 1 4 8 16 17 25 11 5 8 16 17 25 11 7 3 39 29 36 1 4 8 16 17 25 11 7 3 58 46 29 36 1 4 8 16 17 25 11 7 3 59 61 51 52 47 54 47 56 47 56 47 57 58 46 29 36 1 4 8 16 17 24 32 1 4 8 16 17 25 11 7 3 34 1 4 8 16 17 25 11 7 3 35 9 16 17 25 11 7 3 59 60 46 26 30 1 4 8 16 17 24 32 1 4 8 16 19 11 7 3 34 1 4 8 16 19 11 7 3 34 1 4 8 16 19 11 7 3 35 11 7 3 60 46 27 1 4 8 16 17 25 11 7 3 62

Árbol de Análisis Sintáctico:





6.1.3. Fichero Correcto 3

```

1 let string nombre;
2 let string saludo;
3 let int edad;
4 saludo = "Hola, ";
5 read nombre;
6 read edad;
7 if (edad > 18) {
8     write saludo;
9     write nombre;
10 }

```

fichero3.txt

Listado de Tokens:

```

1 <let,>
2 <string,>
3 <identificador,0>
4 <puntoComa,>
5 <let,>
6 <string,>
7 <identificador,1>
8 <puntoComa,>
9 <let,>
10 <int,>
11 <identificador,2>
12 <puntoComa,>
13 <identificador,1>
14 <asignacion,>
15 <cadena,Hola, >
16 <puntoComa,>

```

```

17 <read,>
18 <identificador,0>
19 <puntoComa,>
20 <read,>
21 <identificador,2>
22 <puntoComa,>
23 <if,>
24 <parentesisApertura,>
25 <identificador,2>
26 <mayor,>
27 <entero,18>
28 <parentesisCierre,>
29 <llaveApertura,>
30 <write,>
31 <identificador,1>
32 <puntoComa,>
33 <write,>
34 <identificador,0>
35 <puntoComa,>
36 <llaveCierre,>
37 <eof,>

```

Tabla de Símbolos:

```

1 CONTENIDO DE LA TABLA # 1 :
2 * LEXEMA : 'nombre'
3   +Tipo: cadena
4   +Desplazamiento: 0
5 * LEXEMA : 'saludo'
6   +Tipo: cadena
7   +Desplazamiento: 64
8 * LEXEMA : 'edad'
9   +Tipo: entero
10  +Desplazamiento: 128

```

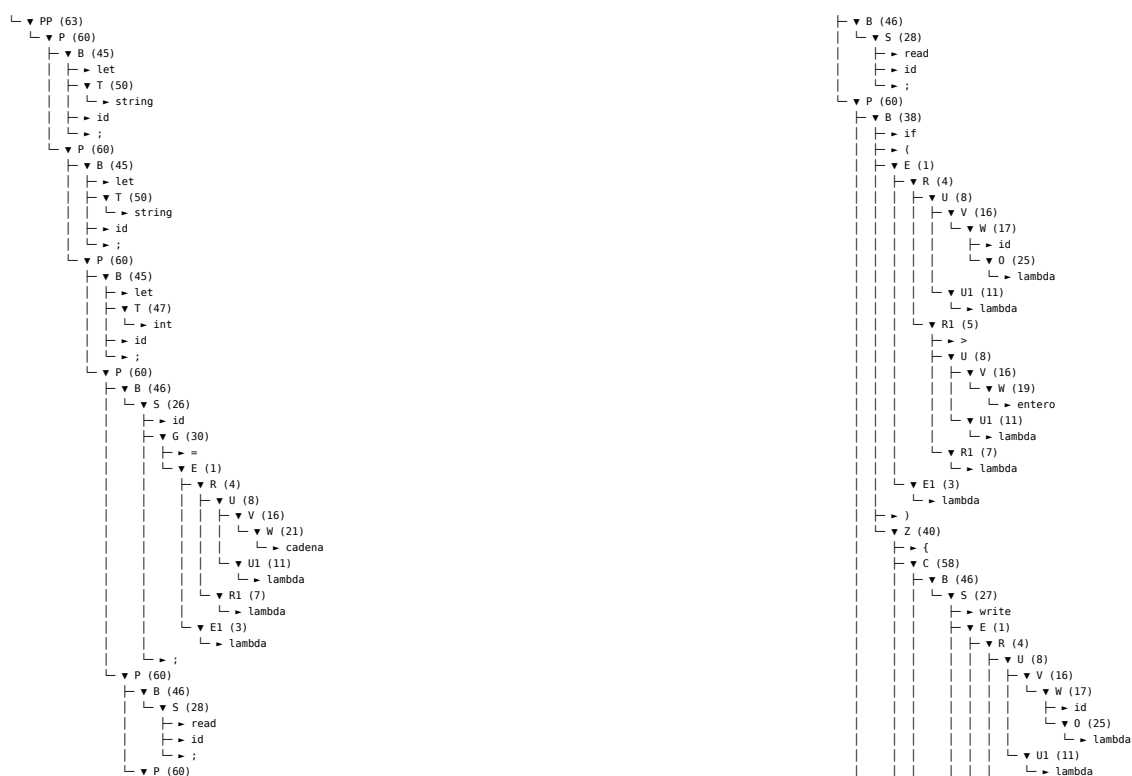
Secuencia de Parse:

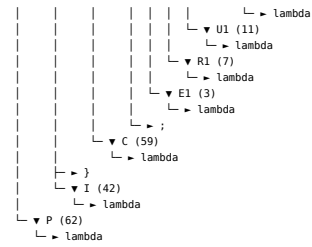
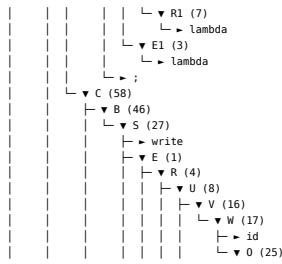
```

1 Descendente
2 63 60 45 50 60 45 50 60 45 47 60 46 26 30 1 4 8 16 21 11 7 3 60 46 28 60 46 28 60 38 1 4 8 16 17 25 11 5 8 16 19 11 7 3
   40 58 46 27 1 4 8 16 17 25 11 7 3 58 46 27 1 4 8 16 17 25 11 7 3 59 42 62

```

Árbol de Análisis Sintáctico:





6.1.4. Fichero Correcto 4

```

1 let int x;
2 let int y;
3 x = 10;
4 y = 3;
5 function void imprimir(int val) {
6     write val;
7     return;
8 }
9 if (x > y && x > 5) {
10     //TODO: Mirar si queremos que esto funcione :--x;
11     let float m;
12 }
13 else{
14     imprimir(x);
15 }
16
17 imprimir(y);

```

fichero4.txt

Listado de Tokens:

```

1 <let,>
2 <int,>
3 <identificador,0>
4 <puntoComa,>
5 <let,>
6 <int,>
7 <identificador,1>
8 <puntoComa,>
9 <identificador,0>
10 <asignacion,>
11 <entero,10>
12 <puntoComa,>
13 <identificador,1>
14 <asignacion,>
15 <entero,3>
16 <puntoComa,>
17 <function,>
18 <void,>
19 <identificador,2>
20 <parentesisApertura,>
21 <int,>
22 <identificador,-1>
23 <parentesisCierre,>
24 <llaveApertura,>
25 <write,>
26 <identificador,-1>
27 <puntoComa,>
28 <return,>
29 <puntoComa,>
30 <llaveCierre,>
31 <if,>
32 <parentesisApertura,>
33 <identificador,0>
34 <mayor,>
35 <identificador,1>

```

```

36 <and,>
37 <identificador,0>
38 <mayor,>
39 <entero,5>
40 <parentesisCierre,>
41 <llaveApertura,>
42 <let,>
43 <float,>
44 <identificador,3>
45 <puntoComa,>
46 <llaveCierre,>
47 <else,>
48 <llaveApertura,>
49 <identificador,2>
50 <parentesisApertura,>
51 <identificador,0>
52 <parentesisCierre,>
53 <puntoComa,>
54 <llaveCierre,>
55 <identificador,2>
56 <parentesisApertura,>
57 <identificador,1>
58 <parentesisCierre,>
59 <puntoComa,>
60 <eof,>

```

Tabla de Símbolos:

```

1 CONTENIDO DE LA TABLA # 1 :
2 * LEXEMA : 'x'
3   +Tipo: entero
4   +Desplazamiento: 0
5 * LEXEMA : 'y'
6   +Tipo: entero
7   +Desplazamiento: 2
8 * LEXEMA : 'imprimir'
9   +Tipo: función
10  +Desplazamiento: 0
11  +TipoRet: void
12  +NumParametros: 1
13  +Parametros: entero
14 * LEXEMA : 'm'
15   +Tipo: real
16   +Desplazamiento: 4
17
18 CONTENIDO DE LA TABLA # 2 :
19 * LEXEMA : 'val'
20   +Tipo: entero
21   +Desplazamiento: 0

```

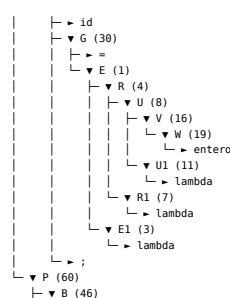
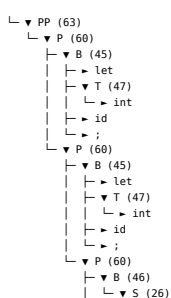
Secuencia de Parse:

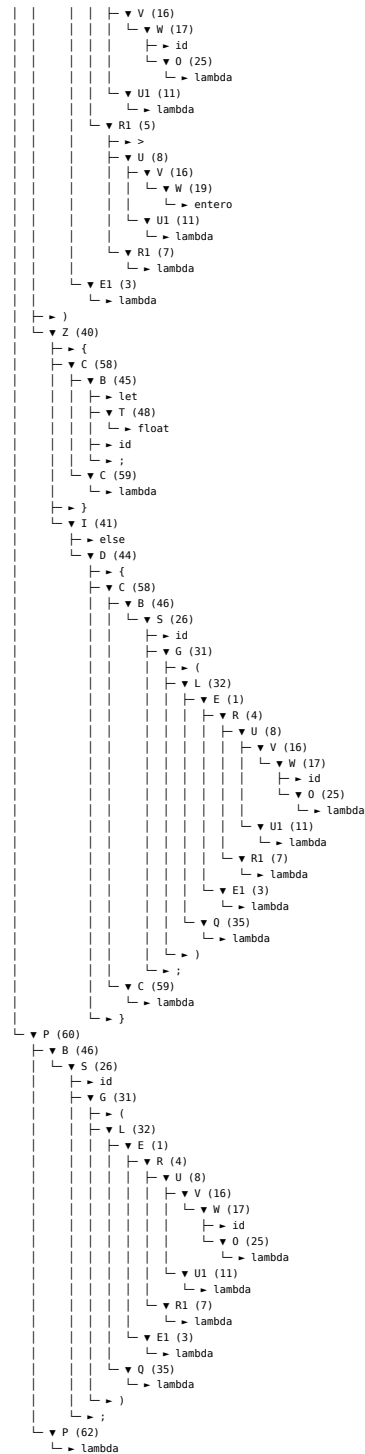
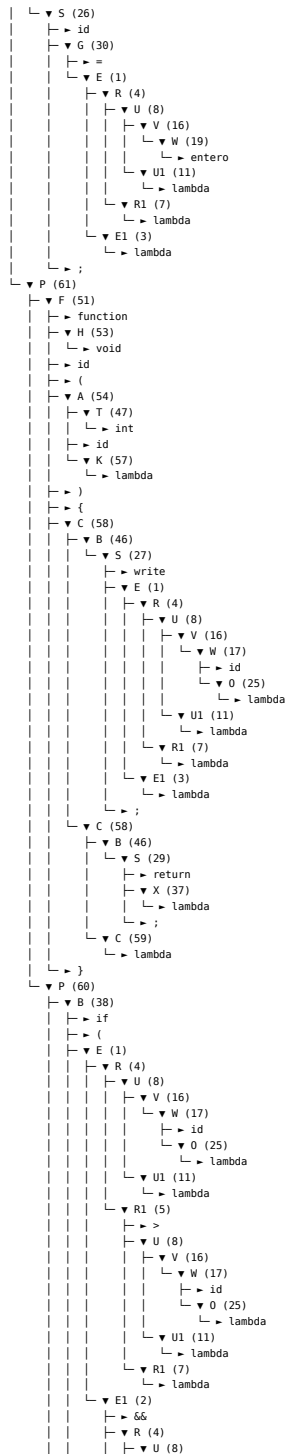
```

1 Descendente
2 63 60 45 47 60 45 47 60 46 26 30 1 4 8 16 19 11 7 3 60 46 26 30 1 4 8 16 19 11 7 3 61 51 53 54 47 57 58 46 27 1 4 8 16
   17 25 11 7 3 58 46 29 37 59 60 38 1 4 8 16 17 25 11 5 8 16 17 25 11 7 2 4 8 16 17 25 11 5 8 16 19 11 7 3 40 58 45
   48 59 41 44 58 46 26 31 32 1 4 8 16 17 25 11 7 3 35 59 60 46 26 31 32 1 4 8 16 17 25 11 7 3 35 62

```

Árbol de Análisis Sintáctico:





6.1.5. Fichero Correcto 5

```

1 let int n;
2 n = 5;
3 function int factorial(int x) {
4     if (x > 1) {
5         return x + factorial(x + -1);
6     }
7     return 1;
8 }
9 write factorial(n);

```

fichero5.txt

Listado de Tokens:

```

1 <let,>
2 <int,>

```

```

3 <identificador,0>
4 <puntoComa,>
5 <identificador,0>
6 <asignacion,>
7 <entero,5>
8 <puntoComa,>
9 <function,>
10 <int,>
11 <identificador,1>
12 <parentesisApertura,>
13 <int,>
14 <identificador,-1>
15 <parentesisCierre,>
16 <llaveApertura,>
17 <if,>
18 <parentesisApertura,>
19 <identificador,-1>
20 <mayor,>
21 <entero,1>
22 <parentesisCierre,>
23 <llaveApertura,>
24 <return,>
25 <identificador,-1>
26 <suma,>
27 <identificador,1>
28 <parentesisApertura,>
29 <identificador,-1>
30 <suma,>
31 <resta,>
32 <entero,1>
33 <parentesisCierre,>
34 <puntoComa,>
35 <llaveCierre,>
36 <return,>
37 <entero,1>
38 <puntoComa,>
39 <llaveCierre,>
40 <write,>
41 <identificador,1>
42 <parentesisApertura,>
43 <identificador,0>
44 <parentesisCierre,>
45 <puntoComa,>
46 <eof,>

```

Tabla de Símbolos:

```

1 CONTENIDO DE LA TABLA # 1 :
2 * LEXEMA : 'n'
3   +Tipo: entero
4   +Desplazamiento: 0
5 * LEXEMA : 'factorial'
6   +Tipo: función
7   +Desplazamiento: 0
8   +TipoRet: entero
9   +NumParametros: 1
10  +Parametros: entero
11
12 CONTENIDO DE LA TABLA # 2 :
13 * LEXEMA : 'x'
14   +Tipo: entero
15   +Desplazamiento: 0

```

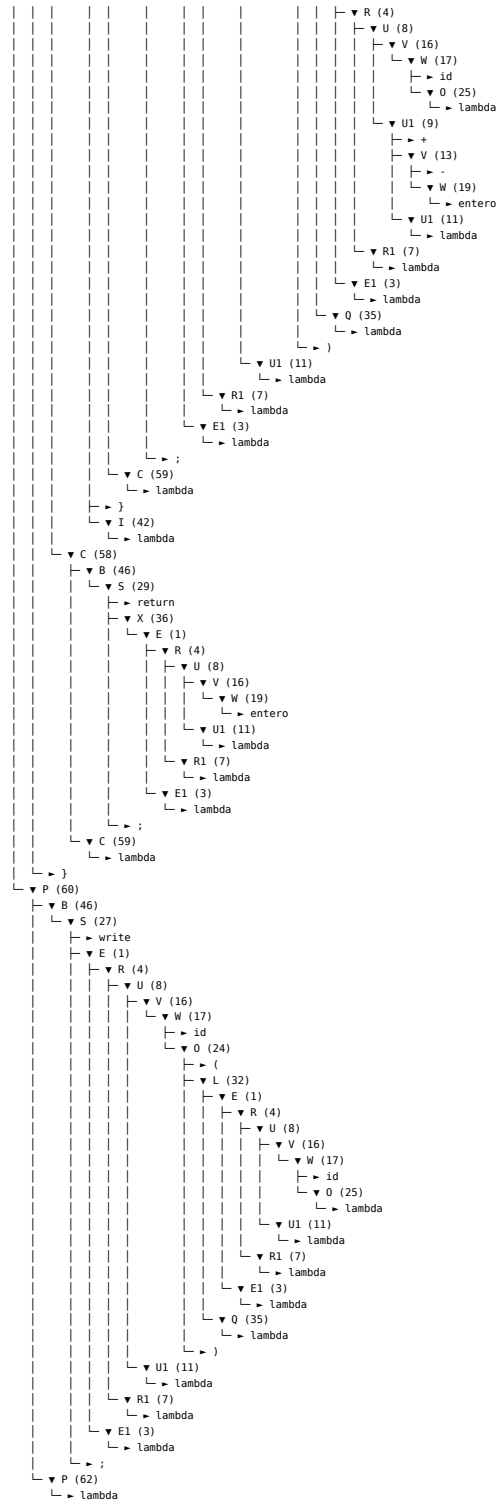
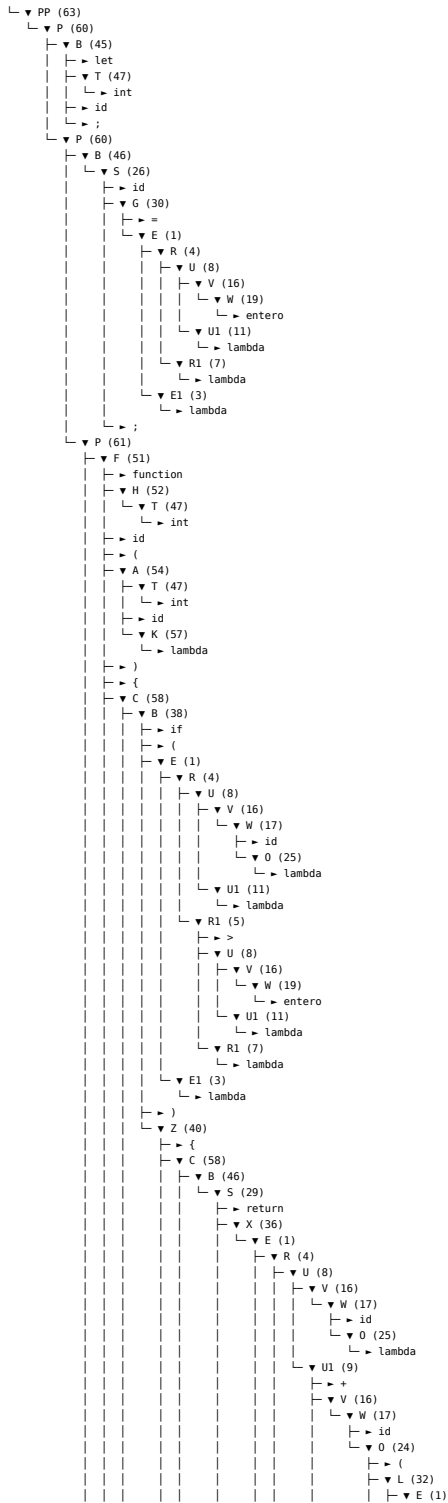
Secuencia de Parse:

```

1 Descendente
2 63 60 45 47 60 46 26 30 1 4 8 16 19 11 7 3 61 51 52 47 54 47 57 58 38 1 4 8 16 17 25 11 5 8 16 19 11 7 3 40 58 46 29 36
   1 4 8 16 17 25 9 16 17 24 32 1 4 8 16 17 25 9 13 19 11 7 3 35 11 7 3 59 42 58 46 29 36 1 4 8 16 19 11 7 3 59 60 46
   27 1 4 8 16 17 24 32 1 4 8 16 17 25 11 7 3 35 11 7 3 62

```

Árbol de Análisis Sintáctico:



6.2. Ficheros con errores

6.2.1. Fichero Erróneo 1

```

1 let int a;
2 let float b;
3 b = 3.14;
4 a = b;
5 write a;

```

fichero1_erroneo.txt

Errores obtenidos:


```
1 === ERRORES ===
2 error en la línea 4: Intentando asignar a una variable de tipo entero un valor de tipo real
```

6.2.2. Fichero Erróneo 2

```
1 let int x;
2 x = 5;
3 if (x + 3) {
4     write x;
5 }
```

fichero2_erroneo.txt

Errores obtenidos:

```
1 === ERRORES ===
2 error en la línea 3: La expresion dentro del if debe ser de tipo logico, sin embargo esta evaluando una expresion de
   tipo entero
```

6.2.3. Fichero Erróneo 3

```
1 let float f;
2 f = 2.5;
3 function int sumar(int a, int b) {
4     return a + b;
5 }
6 write sumar(f, 10);
```

fichero3_erroneo.txt

Errores obtenidos:

```
1 === ERRORES ===
2 error en la línea 6: Los parametros de la llamada a la funcion son incorrectos, se esperaba los tipos entero, entero
3 error en la línea 6: Se estan comparando dos expresiones de distinto tipo, la primera expresion es del tipo error ,
   mientras que la segunda es del tipo void
4 error en la línea 6: Se estan comparando dos expresiones de distinto tipo, la primera expresion es del tipo error,
   mientras que la segunda es del tipo void
```

6.2.4. Fichero Erróneo 4

```
1 function int darReal(void) {
2     let float r;
3     r = 1.5;
4     return r;
5 }
6 write darReal();
```

fichero4_erroneo.txt

Errores obtenidos:

```
1 === ERRORES ===
2 error en la línea 4: Tipo devuelto incorrecto, se esperaba devolver un tipo entero pero se devuelve un tipo real
```

6.2.5. Fichero Erróneo 5

```
1 let int a;
2 let boolean b;
3 a = 5;
4 b = true;
5 write a + b;
```

fichero5_erroneo.txt

Errores obtenidos:

```
1 === ERRORES ===
2 error en la línea 5: Solo se pueden sumar expresiones del tipo entero o real. La primera expresion es del tipo logico,
   mientras que la segunda es del tipo void
3 error en la línea 5: Se estan comparando dos expresiones de distinto tipo, la primera expresion es del tipo error,
   mientras que la segunda es del tipo void
```